YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

Project – High Level Design

on

Honeypot Network for Agriculture Threat Intelligence

SUBMITTED BY

Suhail PM- 23BCCE136

Industry Mentor: Mr.Shashank Kamoor

Guided by: Ms.Vijayalaxmi

DEPARTMENT OF COMPUTER SCIENCE

IN PARTIAL FULLFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF
BACHELOR OF COMPUTER APPLICATION/ BACHELOR OF COMPUTER SCIENCE

May - 2026

TABLE OF CONTENTS

1. Introduction
1.1 Purpose of the Document 3
1.2 Scope 3
1.3 Intended Audience 3

2. Module Design
2.1 Module: honeypot.py 4
2.2 Configuration Constants 4
2.3 Function: log_connection() 4
2.4 Function: handle_connection() 5
2.5 Function: start_honeypot() 6
2.6 Module: test_connection.py 7

3. Web Module Design
3.1 login.html — Authentication Logic 8
3.2 Farmer ID Generation 9
3.3 Event Logging to localStorage 9
3.4 fake-page.html — Honeypot Trap 10
3.5 honeypot_dashboard.html — Monitoring 11
3.6 honeypot.js — Tracker Library 11

4. Data Design
4.1 Data Model 12
4.2 Data Access Mechanism 12
4.3 Data Retention Policies 13
4.4 Data Migration 13

5. Interfaces 14

6. State and Session Management 15

7. Error Handling 16

8. Security Implementation Detail
8.1 Security Aspects 17
8.2 Performance Aspects 18

9. References 18

1. Introduction

1.1 Purpose of the Document

The purpose of this Low Level Design document is to serve as the definitive technical reference for the implementation of the AgriCloud Honeypot system.

It bridges the gap between the architectural overview described in the HLD and the actual written code, providing enough detail that any developer can understand, reproduce, test, or extend any module without needing to read the source code first.

1.2 Scope

This Low Level Design document provides code-level specification for every module in the AgriCloud Honeypot.

It describes exactly how each module is implemented:

- Function signatures and parameters
- Variable definitions and constants
- Socket configuration parameters
- Authentication logic step by step
- localStorage key names
- Error handling per function

Covers: honeypot.py, test_connection.py, login.html, fake-page.html, honeypot_dashboard.html, honeypot.js

1.3 Intended Audience

- Developers implementing or extending the honeypot
- Testers writing test cases against individual functions
- Academic examiners verifying code-level understanding
- Students learning Python socket programming

2. Module Design

2.1 Module: honeypot.py

File : honeypot.py

Language : Python 3.x

Standard Libraries : socket, datetime, threading, os

Entry Point : if __name__ == '__main__':

start_honeypot()

External Dependencies: None — no pip install required

Output File : honeypot_log.txt

2.2 Configuration Constants

HOST = '0.0.0.0'

Accepts connections from any interface.

Change to '127.0.0.1' for localhost only.

PORT = 8888

Honeypot port. Does not need admin privileges.

LOG_FILE = 'honeypot_log.txt'

Resolved to absolute path via os.path.abspath()

2.3 Function: log_connection()

Signature:

```
log_connection(ip_address: str,
               port: int,
               timestamp: str) -> None
```

Parameters:

ip_address String — e.g. '127.0.0.1'

port Integer — always 8888

timestamp String — e.g. '2026-04-17 10:03:29'

Returns None

Side Effect Prints to terminal + appends to LOG_FILE

Implementation logic:

```
log_entry = (
  f'[{timestamp}] CONNECTION ATTEMPT |'
  f'IP: {ip_address:20s} |'
  f'Port: {port} |'
  f'Status: LOGGED & DENIED\n'
)
print(log_entry.strip())
with open(LOG_FILE, 'a') as f:
    f.write(log_entry)
```

Key detail:

:20s pads IP to 20 characters for aligned columns. 'with' statement guarantees file closes even on error.

2.4 Function: handle_connection()

Signature:

```
handle_connection(client_socket: socket.socket,
                  client_address: tuple) -> None
```

Three stages always execute in this order:

```
try:
  STAGE 1 — Deceive
    client_socket.send(FAKE_BANNER.encode('utf-8'))

  STAGE 2 — Log
    log_connection(ip_address, PORT, timestamp)

except Exception as e:
    print(f'[ERROR] {ip_address}: {e}')

finally:
  STAGE 3 — Close (ALWAYS runs even if error occurs)
    client_socket.close()
```

Key detail:

Without finally, a send() error would leave the socket open, consuming a file descriptor until thread exits.

2.5 Function: start_honeypot()

Signature:
start_honeypot() -> None (never returns normally)

Full socket initialisation sequence:

```
STEP 1 — Create TCP socket
  server_socket = socket.socket(
      socket.AF_INET,      # IPv4 addressing
      socket.SOCK_STREAM   # TCP — reliable, ordered
  )

STEP 2 — Allow immediate port reuse after restart
  server_socket.setsockopt(
      socket.SOL_SOCKET,
      socket.SO_REUSEADDR, 1
  )
  Without this: 'Address already in use' error on restart.

STEP 3 — Attach socket to address and port
  server_socket.bind((HOST, PORT))

STEP 4 — Begin listening
  server_socket.listen(5)
  Queues up to 5 pending connections.

STEP 5 — Main accept loop
  while True:
      client_socket, client_address = (
          server_socket.accept()   # BLOCKS until connection
      )
      thread = threading.Thread(
          target=handle_connection,
          args=(client_socket, client_address),
          daemon=True   # Auto-killed on process exit
      )
      thread.start()

CTRL+C handling:
  except KeyboardInterrupt → prints goodbye message finally → server_socket.close() releases port 8888
```

2.6 Module: test_connection.py

```
File     : test_connection.py
Purpose  : Simulate attacker connecting to port 8888
Target   : 127.0.0.1:8888
Timeout  : 5 seconds (prevents script hanging)
Buffer   : recv(1024) — max 1024 bytes received
```

Code flow:

```
sock = socket.socket(AF_INET, SOCK_STREAM)
sock.settimeout(5)
sock.connect((TARGET_HOST, TARGET_PORT))
response = sock.recv(1024).decode(
    'utf-8', errors='replace'
)
print(response)
```

errors='replace' prevents crash on unexpected byte sequences.

3. Web Module Design

3.1 login.html — Authentication Logic

Demo credentials:

Role	**Farm ID**	**Password**
Admin	agri@admin	AgriAdmin@
User	user	user@123
Farmer ID	AC-KL-1142	Set by user
Attacker	Any Admin ID used my Hacker	Any → Trap

Login validation logic:

```
let attempts = 0;

function attemptLogin(farmerId, password) {
  const valid = checkCredentials(farmerId, password);
  // Checks: hardcoded accounts + localStorage farmers

  if (valid) {
    localStorage.setItem('loggedInFarmer', farmerId);
    window.location.href = 'farm-details.html';
  } else {
    attempts++;
    logEvent('LOGIN_ATTEMPT', farmerId);
    updateDots(attempts);  // Fills red dot 1, 2, 3

    if (attempts >= 3) {
      logEvent('ACCESS_DENIED', farmerId);
      showQuarantineOverlay(); // SESSION QUARANTINED
      setTimeout(() => {
        window.location.href = 'fake-page.html';
      }, 2000);  // 2-second delay before redirect
    }
  }
}
```

3.2 Farmer ID Generation

Format: AC-KL/KA-XXXX
AC = Agriculture Farmer Account (fixed)
KL/KA = Based on your State
XXXX = 4-digit random from Math.floor(
Math.random() * 9000) + 1000

Uniqueness check:

```
function isUniqueId(newId) {
  const existing = JSON.parse(
    localStorage.getItem('registered_farmers') || '[]'
  );
  return !existing.some(f => f.farmerId === newId);
}

let id = generateFarmerId();
while (!isUniqueId(id)) id = generateFarmerId();
```

3.3 Event Logging to localStorage

All pages use:

```
function logEvent(event, user) {
  const logs = JSON.parse(
    localStorage.getItem('honeypot_logs') || '[]'
  );
  logs.unshift({
    ts    : new Date().toLocaleString('en-IN'),
    event : event,   // PAGE_VISIT / LOGIN_ATTEMPT / etc.
    user  : user || '—'
  });
  localStorage.setItem('honeypot_logs',
      JSON.stringify(logs));
}
```

unshift() adds to FRONT of array — newest entry first.

3.4 fake-page.html — Honeypot Trap

Feature comparison:

Feature	Real Dashboard	Fake Trap Page
Sensor values	Static demo data	Math.random() ranges
Update interval	Static	setInterval(3000ms)
Action buttons	Real farm actions	PERMISSION DENIED modal
Farmer name	From localStorage	UNAUTHORISED TERMINAL
Intel panel	Not present	Shows attacker's ID

Fake sensor code:

```
function updateFakeSensors() {
  // Soil temp  : 24.0 – 28.0 °C
  (24 + Math.random() * 4).toFixed(1) + ' °C'

  // Moisture   : 62.0 – 70.0 %
  (62 + Math.random() * 8).toFixed(1) + ' %'

  // pH level   : 6.20 – 6.80
  (6.2 + Math.random() * 0.6).toFixed(2)

  // Light      : 8000 – 12000 lux
  Math.round(8000 + Math.random() * 4000) + ' lux'

  // Wind speed : 8.0 – 14.0 km/h
  (8 + Math.random() * 6).toFixed(1) + ' km/h'
}
setInterval(updateFakeSensors, 3000);
```

3.5 honeypot_dashboard.html — Monitoring

Polling mechanism:

```
let lastLogCount = 0;

function pollLogs() {
  try {
    const saved = JSON.parse(
      localStorage.getItem('honeypot_logs') || '[]'
    );
    if (saved.length > lastLogCount) {
      const newEntries = saved.slice(
          0, saved.length - lastLogCount
      );
      newEntries.reverse().forEach(
          e => addEntry(e.event, e.user)
      );
      lastLogCount = saved.length;
    }
  } catch(err) {}
}
setInterval(pollLogs, 800);
```

Attacker → TCP Server → Log File
User → Fake Page → Tracker → localStorage → Dashboard

3.6 honeypot.js — Tracker Library

Fields captured on every page:

ts	ISO timestamp
ts_local	Locale formatted timestamp
url	Full page URL
browser	Chrome / Firefox / Safari / Edge
os	Windows / macOS / Linux / Android / iOS
screen	e.g. 1920x1080
tz	e.g. Asia/Kolkata
touch	true if touchscreen device
ref	Referrer URL or 'Direct'

Events fired:

PAGE_VISIT	On script load (every page)
PAGE_EXIT	On window beforeunload + duration_secs (time on page)
HUMAN_INTERACTION	On first mouse move
HUMAN_INTERACTION	On first scroll + scroll depth %

4. Data Design

4.1 Data Model

honeypot_log.txt — one line per connection:

```
[2026-04-17 10:03:29] CONNECTION ATTEMPT |
IP: 127.0.0.1          | Port: 8888 |
Status: LOGGED & DENIED

timestamp   : strftime('%Y-%m-%d %H:%M:%S')
ip_address  : :20s padded for column alignment
port        : always 8888
status      : always LOGGED & DENIED
```

localStorage['honeypot_logs'] — JSON sample:


```
[
  { "id": 1713340409123,
    "event": "ACCESS_DENIED",
    "user": "hacker123",
    "ts": "17/04/2026, 10:03:29",
    "browser": "Chrome",
    "os": "Windows",
    "screen": "1920x1080" },

  { "id": 1713340405000,
    "event": "LOGIN_ATTEMPT",
    "user": "hacker123" },

  { "id": 1713340400000,
    "event": "PAGE_VISIT",
    "user": "—" }
]
```


4.2 Data Access Mechanism

Python : open(LOG_FILE, 'a') — append mode
'with' statement guarantees file close on error
OS handles atomic writes for short log lines

JS read : JSON.parse(localStorage.getItem(key))

JS write: localStorage.setItem(key, JSON.stringify(arr))
All operations synchronous on main thread

Dashboard: reads localStorage every 800ms
Only new entries processed (delta from lastLogCount)
Reversed before display for chronological order

4.3 Data Retention Policies

honeypot_log.txt
Retained indefinitely. Append-only. No auto-rotation.
Manual deletion required.

honeypot_logs (localStorage)
Until CLEAR button clicked or browser data cleared.
Max 200 entries. Oldest dropped when limit reached.

registered_farmers (localStorage)
Indefinitely until browser data is cleared.
No expiry mechanism.

4.4 Data Migration

No migration needed. Ephemeral lab sessions.
No database schema. No version-controlled format.
Log files can be manually copied for reporting.

5. Interfaces

login.html
User : Attacker / Farmer
Feature : 3-attempt counter, logEvent() on each action,
animated sunrise CSS

farm-details.html
User : Legitimate Farmer
Feature : Reads loggedInFarmer from localStorage,
IoT sensor data display

fake-page.html
User : Attacker
Feature : Math.random() sensors every 3s,
all buttons → PERMISSION DENIED modal

honeypot_dashboard.html
User : SOC Analyst
Feature : pollLogs() every 800ms, conic-gradient gauge,
10-bar activity chart, top IPs panel

honeypot_log.txt
User : Operator
Feature : Aligned plain text, one line per TCP connection

Python Terminal
User : Operator
Feature : Real-time stdout output, mirrors log file

6. State and Session Management

Python: FULLY STATELESS
Each call to handle_connection() is independent.
No shared mutable state between threads.

localStorage key lifecycle:

honeypot_logs
Set by : all pages + honeypot.js
Read by : dashboard pollLogs()
Clear : CLEAR button or browser clear

registered_farmers
Set by : register.html
Read by : login.html checkCredentials()
Clear : browser clear only

loggedInFarmer
Set by : login.html on valid login
Read by : farm-details.html on load
Clear : logout or browser clear

Attacker session full lifecycle:
1 Open login.html. No session key. Counter = 0.
2 Wrong credentials ×3. Overlay shown after 3rd.
3 setTimeout(2000) → window.location.href = fake-page
4 Fake page loads. UNAUTHORISED TERMINAL shown.
5 Sensors randomise. Buttons return DENIED.
6 Tab closed → PAGE_EXIT fired with duration_secs.

7. Error Handling

ConnectionRefusedError
  Module : test_connection.py
  Handling : 'Is honeypot.py running?' message printed

send() fails mid-transmission
  Module : honeypot.py handle_connection
  Handling : except prints error;
          finally always closes socket

File write error (disk full / permissions)
  Module : honeypot.py log_connection
  Handling : Propagates to handle_connection;
          socket still closed in finally

Port already in use on startup
  Module : honeypot.py start_honeypot
  Handling : SO_REUSEADDR prevents most cases;
          OSError shown in terminal

KeyboardInterrupt (Ctrl+C)
  Module : honeypot.py main loop
  Handling : Goodbye message printed;
          server_socket.close() in finally

JSON parse error in localStorage
  Module : honeypot_dashboard.html
  Handling : try/catch silently ignores error;
          dashboard stays functional

localStorage unavailable
  Module : honeypot.js
  Handling : try/catch silently ignores;
          backend POST still attempted

8. Security Implementation Detail

8.1 Security Aspects

WHY FAKE PAGE CANNOT BE DETECTED BY ATTACKER

Visual identity
Identical CSS, layout, and AgriCloud branding.
Side-by-side comparison shows zero difference.

Sensor behaviour
Math.random() within same realistic ranges as real IoT.
Updates every 3 seconds — mimics live data stream.

No network difference
Both pages load from same filesystem origin.
URL pattern is identical from attacker's view.

Only observable tell
Action buttons return PERMISSION DENIED.
But a real restricted system also denies actions —
so even this is believable to an attacker.

WHY HONEYPOT HAS ZERO EXPLOITABLE ATTACK SURFACE

Python socket server
Sends only hardcoded ASCII string.
No input processed, executed, or written to system.
Only side effect is appending to log file.

Web portal
Runs entirely in the browser.
No server-side code, no database, no shell,
no network transmission of real data.

localStorage scope
Confined to file:// origin in the browser.
Not accessible from external scripts or requests.

Farmer ID system
Checked for uniqueness before issuance.
Collision regenerates automatically.
Format AGF-YYYY-XXXX resists obvious guessing.

8.2 Performance Aspects

Threading model
One daemon thread per connection.
Lightweight — one send() + one file write + close.
Approx 8KB stack per thread.

Concurrency limit
listen(5) backlog = 5 pending connections.
Adequate for classroom (1–5 students).
Production: increase to 50–100.

Dashboard polling
setInterval(800ms) = one localStorage read per cycle.
JSON.parse of 200 entries < 1ms on modern hardware.
DOM update is differential — only new rows added.

Log file I/O
open() + write() + close() per connection.
Each OS syscall ≈ 0.1–1ms.
For >100 connections/sec: use buffered logger or DB.

Memory footprint
Minimal — one thread per active connection.
No external JS libraries beyond Google Fonts.

9. References

---

1. Python Software Foundation. socket library.
https://docs.python.org/3/library/socket.html

2. Python Software Foundation. threading library.
https://docs.python.org/3/library/threading.html

3. Python Software Foundation. datetime library.
https://docs.python.org/3/library/datetime.html

4. MDN Web Docs. Window.localStorage.
https://developer.mozilla.org/

5. OWASP Foundation. Honeypots and Deception Technology.
https://owasp.org/